SI

ShopIntel

AI-Powered Shopify Command Center

Complete Technical Documentation — Architecture, Security, Infrastructure, API Reference, and Deployment Guide

Version: 1.0.0

Framework: Next.js 16.2 / React 19 / TypeScript 5

AI Model: OpenAI GPT-5.4 via Vercel AI SDK v6

Database: PostgreSQL on Railway / Prisma 7.5

Deployment: Vercel (Edge + Serverless)

Prepared by: Growth Capital Engineering

Date: March 2026

Table of Contents

1 Executive Overview

2 System Architecture

2.1 High-Level Architecture

2.2 Request Flow

2.3 Technology Stack

3 Project Structure

4 Authentication & Security

4.1 JWT Authentication

4.2 Password Hashing

4.3 Middleware & Security Headers

4.4 Rate Limiting

4.5 Role-Based Access Control

5 Database & Railway

5.1 Schema Design

5.2 Prisma ORM

5.3 Railway Configuration

6 API Reference

6.1 Auth Endpoints

6.2 Chat Endpoint

6.3 Conversation Endpoints

7 AI Agent System

7.1 System Prompt

7.2 Tool Definitions

7.3 Streaming & Generative UI

8 Shopify Integration

8.1 GraphQL Client

8.2 Query Functions

8.3 Analytics Engine

8.4 Trend Forecasting

8.5 Theme Deployment

9 Frontend & Components

9.1 Dashboard Layout

9.2 Generative UI Components

9.3 Chat Interface

9.4 Liquid Preview System

10 Vercel Deployment

10.1 Build Pipeline

10.2 Environment Variables

10.3 Runtime Configuration

11 Environment Variables Reference

12 Demo Credentials & Quick Start

1. Executive Overview

ShopIntel is an autonomous AI agent for Shopify merchants, built as a full-stack Next.js application. It combines a real-time analytics dashboard with a natural-language chat interface powered by GPT-5.4. The AI agent doesn't just answer questions — it proactively analyzes store data, predicts trends, generates production-grade Shopify Liquid code, and deploys it directly to live themes.

Key Capabilities

- **Natural Language Querying** — Ask "How are sales?" and get interactive charts, KPIs, and forecasts
- **Generative UI** — AI selects and renders the right React component (charts, tables, grids) in real-time
- **Trend Forecasting** — WoW/MoM growth, anomaly detection, product-level trends
- **Liquid Code Generation** — AI writes complete Shopify sections with preview
- **One-Click Deploy** — Push generated code directly to the live Shopify theme
- **Persistent Conversations** — Chat history saved to PostgreSQL, restored on refresh

Architecture Summary

```
Browser  ──>  Vercel (Next.js 16)  ──>  OpenAI GPT-5.4  ──>  Shopify Admin API
                        │
                        └──>  Railway PostgreSQL (Prisma 7.5)
```

2. System Architecture

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          CLIENT (Browser)                            │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐       │
│  │  Dashboard   │  │  Chat UI     │  │  Generative Components │     │
│  │  (Analytics) │  │  (useChat)   │  │  (Charts, Tables, etc) │     │
│  └──────┬───────┘  └──────┬───────┘  └──────────┬──────────────┘     │
│         └─────────────────┼──────────────────────┘                    │
│                           │                                           │
│                           │ HTTP/Streaming                            │
└───────────────────────────┼──────────────────────────────────────────┘
                            │
┌───────────────────────────┼──────────────────────────────────────────┐
│                   VERCEL (Next.js 16)                                 │
│                           │                                           │
│  ┌────────────────────────┼──────────────────────────────────┐       │
│  │                  Proxy Middleware                         │       │
│  │          JWT Verification + Security Headers              │       │
│  └────────────────────────┼──────────────────────────────────┘       │
│                           │                                           │
│  ┌──────────┐  ┌──────────┼──────────┐  ┌──────────────────┐        │
│  │ Auth API │  │    Chat API         │  │ Conversation API │        │
│  │ /login   │  │ streamText() +      │  │ CRUD + Messages  │        │
│  │ /logout  │  │ Tool Execution      │  │                  │        │
│  │ /me      │  │                     │  │                  │        │
│  └──────────┘  └─────────────────────┘  └──────────┬───────┘        │
│                           │                         │                 │
└───────────────────────────┼─────────────────────────┼─────────────────┘
                            │                         │
                            │                         │
                            │                         │
            ┌───────────────┼──────────┐              │
            │               │          │              │
  ┌─────────▼─────────┐ ┌───▼──────┐   │   ┌──────────▼──────────┐
  │ OpenAI GPT-5.4    │ │Shopify API │   │   │ Railway Postgres    │
  │ (Tool Calling)    │ │ (GraphQL)  │   │   │ (Prisma 7.5)        │
  └───────────────────┘ └────────────┘   │   └─────────────────────┘
                                         │
                                  ┌──────▼──────┐
                                  │ Shopify Theme │
                                  │ (Asset Deploy)│
                                  └──────────────┘
```

2.2 Request Flow — AI Chat

1. User types message in chat input
2. `usePersistedChat` hook creates/loads conversation in PostgreSQL
3. Message sent to `/api/chat` as streaming POST request
4. Proxy middleware verifies JWT session token
5. API converts messages, calls `streamText()` with GPT-5.4 + tool definitions
6. Model decides which tools to call (getAnalytics, getProducts, etc.)
7. Tools execute Shopify GraphQL queries, return structured data

8. Results stream back as UI message chunks
9. `ToolResultRenderer` maps tool names to React components (charts, tables, grids)
10. Components render in real-time as data arrives
11. On completion, assistant message saved to PostgreSQL

2.3 Technology Stack

LAYER	TECHNOLOGY	VERSION
Framework	Next.js (App Router)	16.2.0
Runtime	React	19.2.4
Language	TypeScript	5.x
AI SDK	Vercel AI SDK	6.0.116
LLM	OpenAI GPT-5.4	2026-03-05
Database	PostgreSQL via Railway	16
ORM	Prisma	7.5.0
Auth	jose (JWT) + bcryptjs	5.x / 3.x
Styling	Tailwind CSS	4.x
Components	shadcn/ui + Recharts	Latest
External API	Shopify Admin GraphQL	2024-10
Hosting	Vercel	Pro

3. Project Structure

```
shopintel/ ├── .env.local # Runtime environment variables ├── next.config.ts # Security headers ├── prisma.config.ts # Prisma adapter config ├── tsconfig.json # TypeScript config ├── postcss.config.mjs # Tailwind CSS v4 ├── package.json # Dependencies + scripts │ ├── prisma/ │ ├── schema.prisma # User, Conversation, Message models │ └── migrations/ # Database migrations │ └── src/ ├── proxy.ts # Auth middleware + security headers │ ├── app/ │ ├── layout.tsx # Root layout (fonts, metadata) │ ├── page.tsx # Dashboard (server component) │ ├── login/ │ │ └── page.tsx # Login page with demo buttons │ └── api/ │ ├── auth/ │ │ ├── login/route.ts │ │ ├── logout/route.ts │ │ └── me/route.ts │ ├── chat/ │ │ └── route.ts # AI streaming endpoint │ └── conversations/ │ ├── route.ts # GET list / POST create │ └── [id]/ │ ├── route.ts # GET / DELETE conversation │ └── messages/ │ └── route.ts # POST message │ ├── components/ │ ├── dashboard.tsx # Main dashboard + chat overlay │ ├── vanta-background.tsx # Animated Three.js background │ ├── chat/ │ │ ├── chat-interface.tsx # Full chat with sidebar │ │ └── tool-result-renderer.tsx # Maps tools to components │ ├── generative/ # AI-rendered data components │ │ ├── kpi-cards.tsx │ │ ├── revenue-chart.tsx │ │ ├── top-products-chart.tsx │ │ ├── status-chart.tsx │ │ ├── orders-table.tsx │ │ ├── customer-list.tsx │ │ ├── inventory-table.tsx │ │ ├── product-grid.tsx │ │ ├── liquid-preview.tsx │ │ └── trend-forecast.tsx │ └── ui/ # shadcn/ui primitives │ ├── lib/ │ ├── auth.ts # JWT + password hashing │ ├── db.ts # Prisma singleton client │ ├── env.ts # Zod env validation │ ├── utils.ts # cn() class merger │ ├── hooks/ │ │ └── use-persisted-chat.ts # Chat with DB persistence │ ├── shopify/ │ │ ├── client.ts # GraphQL client │ │ ├── queries.ts # All Shopify queries │ │ └── types.ts # TypeScript interfaces │ └── ai/ │ ├── tools.ts # 8 tool definitions │ ├── system-prompt.ts # Agent personality + rules │ └── liquid-templates.ts # Liquid section templates │ └── generated/prisma/ # Auto-generated Prisma client
```

4. Authentication & Security

4.1 JWT Authentication

Authentication is implemented via JSON Web Tokens (JWT) using the `jose` library. Sessions are stateless — the JWT contains the full user identity and is signed with `HS256`.

SESSION LIFECYCLE

1. **Login:** User submits email + password to `/api/auth/login`
2. **Verification:** Password checked against bcryptjs hash (or legacy SHA256 with auto-upgrade)
3. **Token Creation:** JWT signed with `JWT_SECRET`, containing `{id, email, name, role}`
4. **Cookie:** Token stored as `session` cookie with security flags
5. **Verification:** Every request verified by proxy middleware
6. **Expiration:** Tokens expire after 24 hours

COOKIE CONFIGURATION

```
{
  httpOnly: true,       // Not accessible via JavaScript
  secure: true,         // HTTPS only
  sameSite: "strict",   // No cross-site sending
  maxAge: 86400,        // 24 hours
  path: "/"             // Available site-wide
}
```

SESSION INTERFACE

```
interface SessionUser {
  id: string;       // CUID from database
  email: string;    // User email
  name: string;     // Display name
  role: "admin" | "client";
}
```

4.2 Password Hashing

Passwords are hashed with **bcryptjs** using 12 salt rounds. The system supports legacy SHA256 hashes and auto-upgrades them on successful login.

```
// Hashing (on registration)
const hash = await bcrypt.hash(password, 12);

// Verification (on login)
const valid = await bcrypt.compare(password, storedHash);

// Legacy migration: if hash doesn't start with "$2", try SHA256
// On success, auto-upgrade to bcryptjs hash
```

4.3 Middleware & Security Headers

File: `src/proxy.ts` — Next.js 16 proxy middleware that intercepts every request.

AUTHENTICATION FLOW

```
Request
  │
  ├── Is /login or /api/auth/*?  →  ALLOW (public)
  │
  ├── Has session cookie?
  │     No  →  Redirect to /login
  │     Yes →  Verify JWT
  │              Invalid →  Delete cookie + redirect
  │              Valid   →  Apply security headers + proceed
  │
  └── Response with security headers
```

SECURITY HEADERS APPLIED

HEADER	VALUE	PURPOSE
X-Content-Type-Options	nosniff	Prevent MIME sniffing
X-Frame-Options	DENY	Prevent clickjacking
X-XSS-Protection	1; mode=block	XSS filter
Referrer-Policy	strict-origin-when-cross-origin	Limit referrer leakage
Permissions-Policy	camera=(), microphone=(), geolocation=()	Disable device APIs
Strict-Transport-Security	max-age=31536000; includeSubDomains	Force HTTPS for 1 year

4.4 Rate Limiting

ENDPOINT	LIMIT	WINDOW	LOCKOUT
`/api/auth/login`	5 attempts per email	15 minutes	15-minute cooldown
`/api/chat`	10 requests per user	1 minute	Rate limit response (429)

> **Note:**
> Rate limiting uses in-memory Maps. In a multi-instance deployment, consider Redis-backed rate limiting for consistency across instances.

4.5 Role-Based Access Control

FEATURE	ADMIN	CLIENT
Dashboard KPIs & Charts	Yes	Yes
Orders Table	Yes	No
Customer List	Yes	No
Inventory Table	Yes	No
AI Chat Agent	Yes	No
Liquid Code Generation	Yes	No
Theme Deployment	Yes	No
Quick Action Buttons	Yes	No

5. Database & Railway

5.1 Schema Design

Three models with simple relational structure. Messages cascade-delete when a conversation is removed.

```
┌──────────────┐        ┌──────────────────┐        ┌──────────────┐
│    User      │        │  Conversation    │        │   Message    │
├──────────────┤        ├──────────────────┤        ├──────────────┤
│ id      (PK) │──┐     │ id          (PK) │──┐     │ id      (PK) │
│ email   (UQ) │  │     │ title            │  │     │ role         │
│ password     │  └────>│ userId      (FK) │  └────>│ conversationId│
│ name         │        │ createdAt        │        │ content      │
│ role         │        │ updatedAt        │        │ createdAt    │
│ createdAt    │        └──────────────────┘        └──────────────┘
└──────────────┘          1:many                      1:many
                                                     (cascade delete)
```

PRISMA SCHEMA

```
model User {
  id            String         @id @default(cuid())
  email         String         @unique
  password      String
  name          String
  role          String         @default("client")
  createdAt     DateTime       @default(now())
  conversations Conversation[]
}

model Conversation {
  id        String    @id @default(cuid())
  title     String    @default("New Conversation")
  userId    String
  user      User      @relation(fields: [userId], references: [id])
  messages  Message[]
  createdAt DateTime  @default(now())
  updatedAt DateTime  @updatedAt
}

model Message {
  id             String       @id @default(cuid())
  conversationId String
  conversation   Conversation @relation(fields: [conversationId],
                               references: [id], onDelete: Cascade)
  role           String       // "user" | "assistant"
  content        String       // JSON-serialized message parts
  createdAt      DateTime     @default(now())
}
```

5.2 Prisma ORM Configuration

Prisma 7.5 with the `@prisma/adapter-pg` for native PostgreSQL connection pooling. The client is generated to `src/generated/prisma/` and instantiated as a singleton.

```
// prisma.config.ts
import path from "node:path";
import { PrismaPg } from "@prisma/adapter-pg";
import { defineConfig } from "prisma/config";

export default defineConfig({
  earlyAccess: true,
  schema: path.join(__dirname, "prisma/schema.prisma"),
  migrate: { adapter: () => new PrismaPg({ connectionString: process.env.DATABASE_URL! }) },
});

// src/lib/db.ts — Singleton pattern
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };
export const prisma = globalForPrisma.prisma ?? new PrismaClient({
  adapter: new PrismaPg({ connectionString: process.env.DATABASE_URL! }),
});
if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

5.3 Railway Configuration

Railway PostgreSQL Instance

- **Provider:**
  Railway (railway.app)
- **Engine:**
  PostgreSQL 16
- **Connection:**
  Via proxy URL (`ballast.proxy.rlwy.net:24687`)
- **Auth:**
  Username/password in connection string
- **Pooling:**
  Handled by Prisma PrismaPg adapter
- **Backups:**
  Railway automatic daily backups

Connection String Format:

```
postgresql://<user>:<password>@ballast.proxy.rlwy.net:24687/railway
```

Important:

The DATABASE_URL must be set in both `.env.local` (local dev) and Vercel environment variables (production). Railway provides the connection string in the dashboard under your PostgreSQL service settings.

6. API Reference

6.1 Auth Endpoints

POST /API/AUTH/LOGIN

FIELD	TYPE	VALIDATION
email	string	Valid email, max 255 chars
password	string	Min 1, max 128 chars

Success (200): `{ user: { id, email, name, role } }` + sets `session` cookie

Errors: 400 (validation), 401 (invalid credentials), 429 (rate limited)

POST /API/AUTH/LOGOUT

Success (200): `{ ok: true }` — Deletes session cookie

GET /API/AUTH/ME

Success (200): `{ user: SessionUser | null }`

6.2 Chat Endpoint

POST /API/CHAT

> **Streaming Response:**
> This endpoint returns a Server-Sent Events stream via the Vercel AI SDK. It does not return JSON.

FIELD	TYPE	CONSTRAINTS
messages	UIMessage[]	Max 50 messages

Auth: Required (session cookie)

Rate Limit: 10 requests/minute per user

Max Duration: 120 seconds

Model: gpt-5.4

Max Tool Steps: 5 (via `stopWhen: stepCountIs(5)`)

6.3 Conversation Endpoints

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/conversations	Required	List user's conversations (newest first)
POST	/api/conversations	Required	Create conversation (body: `{ title? }`)
GET	/api/conversations/[id]	Required + Owner	Get conversation with messages
DELETE	/api/conversations/[id]	Required + Owner	Delete conversation (cascades messages)
POST	/api/conversations/[id]/messages	Required + Owner	Add message (body: `{ role, content }`)

Ownership Check:

All conversation endpoints verify the conversation belongs to the authenticated user (403 Forbidden if not).

7. AI Agent System

7.1 System Prompt

The AI agent is configured as a **sharp, strategic commerce advisor** with specific behavioral rules:

- **Proactive:** Surface anomalies and opportunities unprompted
- **Tool Chaining:** "How's business?" triggers getAnalytics + forecastTrends + getInventory
- **Action-Oriented:** Generate Liquid code AND offer to deploy immediately
- **Predictive:** Always frame data with growth trends and forecasts
- **Concise:** Let the UI components do the heavy lifting — text should be scannable

LIQUID GENERATION DESIGN DIRECTION

When generating Shopify Liquid sections, the AI follows a **luxury clean beauty aesthetic** (Kosas-inspired):

- **Typography:** Founders Grotesk headings, Roboto body, generous letter-spacing
- **Palette:** Black/white primary, dusty rose (#D33167) accent
- **Spacing:** 80-100px vertical padding, 24-32px grid gaps
- **Motion:** Subtle 0.3s transitions, hover scale 1.02-1.05
- **Buttons:** Solid black, uppercase, minimal radius, rose hover

7.2 Tool Definitions

Eight tools are available to the AI agent, defined with Zod schemas for type-safe validation:

TOOL	PURPOSE	KEY PARAMETERS	UI COMPONENT
`getProducts`	Fetch product catalog	searchQuery?, limit? (1-50)	ProductGrid
`getOrders`	Fetch order history	searchQuery?, limit? (1-100)	OrdersTable
`getCustomers`	Fetch customer data	searchQuery?, limit? (1-100)	CustomerList
`getAnalytics`	Compute store metrics	dateFrom?, dateTo?	KPICards + Charts
`getInventory`	Check stock levels	lowStockThreshold?, limit?	InventoryTable
`forecastTrends`	Predict growth patterns	(none)	TrendForecast
`generateLiquid`	Create Shopify section	componentType, code	LiquidPreview
`deploySection`	Push code to theme	sectionName, code	Deploy Alert

7.3 Streaming & Generative UI

The chat uses the Vercel AI SDK's streaming protocol. As the model generates text and tool calls, they're streamed to the client as UI message chunks. The `ToolResultRenderer` component maps each tool's output to the appropriate React component:

```
// Tool name → Component mapping
switch (toolName) {
  case "getProducts":   return <ProductGrid products={result.products} />
  case "getOrders":     return <OrdersTable orders={result.orders} />
  case "getCustomers":  return <CustomerList customers={result.customers} />
  case "getAnalytics":  return <KPICards ... /> + <RevenueChart ... />
  case "getInventory":  return <InventoryTable ... />
  case "forecastTrends": return <TrendForecast data={result} />
  case "generateLiquid": return <LiquidPreview code={result.code} ... />
  case "deploySection":  return <DeployAlert ... />
}
```

8. Shopify Integration

8.1 GraphQL Client

All Shopify communication uses the **Admin GraphQL API (version 2024-10)** via a lightweight custom client.

```
// src/lib/shopify/client.ts
async function shopifyGraphQL<T>(query: string, variables?: Record<string, unknown>): Promise<T>

// Endpoint: https://{SHOPIFY_STORE_URL}/admin/api/2024-10/graphql.json
// Auth Header: X-Shopify-Access-Token: {SHOPIFY_ACCESS_TOKEN}
```

8.2 Query Functions

FUNCTION	GRAPHQL QUERY	DATA RETURNED
`getProducts()`	products(first: N, query: Q)	Title, handle, vendor, status, variants (5), featured image, price range, inventory
`getOrders()`	orders(first: N, query: Q, sortKey: CREATED_AT)	Name, dates, financial/fulfillment status, customer, line items, total prices
`getCustomers()`	customers(first: N, query: Q)	Name, email, phone, order count, lifetime spend, address
`getInventory()`	products(first: N) with variants	Per-variant inventory quantities, SKUs, low-stock flags

8.3 Analytics Engine

Analytics are computed server-side from raw order data (not Shopify's analytics API):

- **Revenue:** Sum of `totalPriceSet.shopMoney.amount` across all orders
- **AOV:** Total revenue / order count
- **Top Products:** Aggregated from line items, sorted by revenue (top 10)
- **Revenue Timeline:** Daily aggregation from order `createdAt`
- **Status Breakdown:** Count of orders per `displayFulfillmentStatus`

8.4 Trend Forecasting

The `forecastTrends()` function provides predictive analytics:

- **WoW Growth:** Compares last 7 days revenue vs prior 7 days
- **MoM Growth:** Compares last 30 days vs prior 30 days
- **Product Trends:** Top 5 growing + top 5 declining products (by revenue change)
- **Anomaly Detection:** Flags daily revenue >2 standard deviations from 30-day mean
- **Statistics:** 30-day mean and standard deviation

8.5 Theme Deployment

The `deploySection` tool pushes Liquid code directly to the active Shopify theme:

1. Fetches the main theme ID via `themes(first:1, roles:MAIN)`
2. Upserts the file at `sections/{sectionName}.liquid`
3. Returns success status with filename

> **Production Warning:**
> The deploy tool writes directly to the live theme. In production, consider adding a staging theme workflow or confirmation step before deploying to the main theme.

9. Frontend & Components

9.1 Dashboard Layout

The main dashboard (`src/components/dashboard.tsx`) is a server-rendered page that fetches all initial data in parallel, then renders an interactive client component with:

- **Header:** Logo, branding, quick-action suggestion buttons (admin only), user info, logout
- **KPI Section:** 4-column metric cards (revenue, orders, AOV, low stock)
- **Charts Row:** Asymmetric 12-col grid — Revenue (5-col), Top Products (4-col), Status (3-col)
- **Data Tables:** Orders (8-col) + Customers (4-col), admin-only
- **Inventory Table:** Full-width with low-stock highlighting, admin-only
- **Chat Overlay:** Minimizable floating card with AI conversation, admin-only
- **Sticky Input Bar:** Fixed bottom input for AI chat, admin-only

9.2 Generative UI Components

COMPONENT	RENDERS WHEN	FEATURES
KPICards	getAnalytics returns	4 metric cards with formatted currency, animated counters
RevenueChart	getAnalytics returns	Recharts line chart with custom tooltip, responsive
TopProductsChart	getAnalytics returns	Recharts bar chart, truncated names with full tooltip
StatusChart	getAnalytics returns	Recharts pie chart for fulfillment status
OrdersTable	getOrders returns	Sortable table with status badges, customer info
CustomerList	getCustomers returns	Sorted by lifetime value, avatar initials
InventoryTable	getInventory returns	Expandable rows, low-stock red highlighting
ProductGrid	getProducts returns	Image grid with vendor, price, inventory count
LiquidPreview	generateLiquid returns	Code view + live iframe preview + copy + deploy
TrendForecast	forecastTrends returns	Growth metrics, trending products, anomaly flags

9.3 Chat Interface

Two chat implementations exist:

1. **Dashboard Overlay** (`dashboard.tsx`) — Floating card on dashboard, minimizable, with sticky input bar
2. **Full Chat** (`chat/chat-interface.tsx`) — Standalone page with conversation sidebar, history management

Both use the `usePersistedChat` hook which wraps `useChat` from the Vercel AI SDK with automatic conversation creation, message persistence to PostgreSQL, and conversation restore on page load.

9.4 Liquid Preview System

The `LiquidPreview` component renders generated Liquid code in two views:

- **Code Tab:** Full Liquid source with copy button
- **Preview Tab:** Sandboxed iframe with product data injected, rendering the HTML/CSS output
- **Deploy Button:** One-click push to Shopify theme via `deploySection` tool

10. Vercel Deployment

10.1 Build Pipeline

```
# package.json scripts
{
  "build": "prisma generate && next build",
  "postinstall": "prisma generate"
}

# Build steps on Vercel:
# 1. npm install (triggers postinstall → prisma generate)
# 2. npm run build
#    a. prisma generate  → creates Prisma client in src/generated/prisma/
#    b. next build       → compiles Next.js (Turbopack)
# 3. Deploy to Vercel edge network
```

10.2 Environment Variables

These must be configured in the Vercel dashboard under **Settings → Environment Variables**:

VARIABLE	REQUIRED	DESCRIPTION
`DATABASE_URL`	Required	PostgreSQL connection string from Railway
`SHOPIFY_STORE_URL`	Required	Store domain (e.g., `mystore.myshopify.com`)
`SHOPIFY_ACCESS_TOKEN`	Required	Shopify Admin API access token
`OPENAI_API_KEY`	Required	OpenAI API key for GPT-5.4
`JWT_SECRET`	Required	Min 32 chars, used for signing session tokens

Security:

Never commit `.env.local` to version control. The `.gitignore` should exclude all `.env*` files. Only set secrets through the Vercel dashboard or CLI.

10.3 Runtime Configuration

SETTING	VALUE	NOTES
Node.js Version	18+	Required for Next.js 16
Max Function Duration	120s	Set via `export const maxDuration = 120` in chat route
Framework Preset	Next.js	Auto-detected by Vercel
Build Command	`prisma generate && next build`	Custom build with Prisma
Output Directory	`.next`	Default Next.js output

11. Environment Variables Reference

VARIABLE	EXAMPLE VALUE	WHERE SET	DESCRIPTION
`DATABASE_URL`	`postgresql://user:pass@host:port/db`	Railway → Vercel	PostgreSQL connection string. Get from Railway dashboard under your Postgres service. Must include username, password, host, port, and database name.
`SHOPIFY_STORE_URL`	`mystore.myshopify.com`	Vercel + .env.local	Your Shopify store's myshopify.com domain. Do NOT include `https://` prefix.
`SHOPIFY_ACCESS_TOKEN`	`shpca_xxxxx`	Vercel + .env.local	Shopify Admin API access token. Created in Shopify Partners → Apps → your custom app → API credentials. Requires: read_products, read_orders, read_customers, write_themes scopes.
`OPENAI_API_KEY`	`sk-proj-xxxxx`	Vercel + .env.local	OpenAI API key with access to gpt-5.4 model. Get from platform.openai.com → API Keys.
`JWT_SECRET`	`64-char hex string`	Vercel + .env.local	Secret for signing JWT session tokens. Generate with: `openssl rand -hex 32`.

			Must be at least 32 characters.

GENERATING SECRETS

```
# Generate JWT_SECRET
openssl rand -hex 32
# or
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

12. Demo Credentials & Quick Start

Demo Accounts

ROLE	EMAIL	PASSWORD	ACCESS LEVEL
Admin	`admin@growthcapital.com`	`growth2026`	Full access: Dashboard, AI Chat, Liquid generation, Deploy, all data
Client	`client@glowpure.com`	`glowpure2026`	Read-only: Dashboard KPIs and charts only

Demo Only:

These credentials are hardcoded for demonstration purposes. In production, implement a proper user registration flow and remove hardcoded accounts.

Quick Start (Local Development)

```
# 1. Clone and install
git clone https://github.com/magnacollective/shopintel.git
cd shopintel
npm install

# 2. Set up environment
cp .env.example .env.local
# Edit .env.local with your credentials

# 3. Set up database
npx prisma generate
npx prisma db push

# 4. Seed demo users (if needed)
# Use the login API or create users via Prisma Studio:
npx prisma studio

# 5. Run development server
npm run dev
# Open http://localhost:3000
```

Shopify App Scopes Required

When creating the Shopify custom app, ensure these **Admin API scopes** are granted:

- `read_products` — Product catalog access
- `read_orders` — Order history access
- `read_customers` — Customer data access
- `write_themes` — Theme file deployment (for Liquid sections)
- `read_inventory` — Inventory level access

ShopIntel — AI-Powered Shopify Command Center
Growth Capital Engineering — March 2026
This document is confidential and intended for internal use only.